DATA-260 Homework 6: Persistent Memory Systems

Student: Sanjana Thummalapalli
SID4: 7801
Verify Seed: 267801
Domain: Clinical Trial Listings
Local Model: qwen3:4b
Github: https://github.com/sanjana-glitch-art/data260-7801

Introduction

This homework extends the Clinical Trial Listing application with persistent document storage and multiple forms of agent memory. MongoDB stores clinical-trial documents, conversation messages, summaries, and episodic memories. A second file-based memory system uses AGENT.md and MEMORY.md to preserve durable project instructions and important conversational facts.

The assignment also compares multiple short-term memory configurations to measure the relationship between prompt size, latency, estimated cost, and memory-recall quality.

Part 1: MongoDB Document Storage

A MongoDB 7 container was configured through Docker Compose. The application connects to the s7801_doc database and stores clinical-trial records in the trial_documents collection.
The collection validator enforces required fields, field types, title length, allowed clinical-trial phases, allowed recruitment statuses, date fields, and rejection of unexpected properties.

The following FastAPI endpoints provide full CRUD behavior:

Method	Endpoint	Purpose
POST	/api/document-trials	Create a clinical-trial document
GET	/api/document-trials	Retrieve all documents
GET	/api/document-trials/{document_id}	Retrieve one document
PUT	/api/document-trials/{document_id}	Update one document
DELETE	/api/document-trials/{document_id}	Delete one document

Valid clinical-trial documents were accepted. Invalid values and documents that violated the MongoDB schema were rejected.

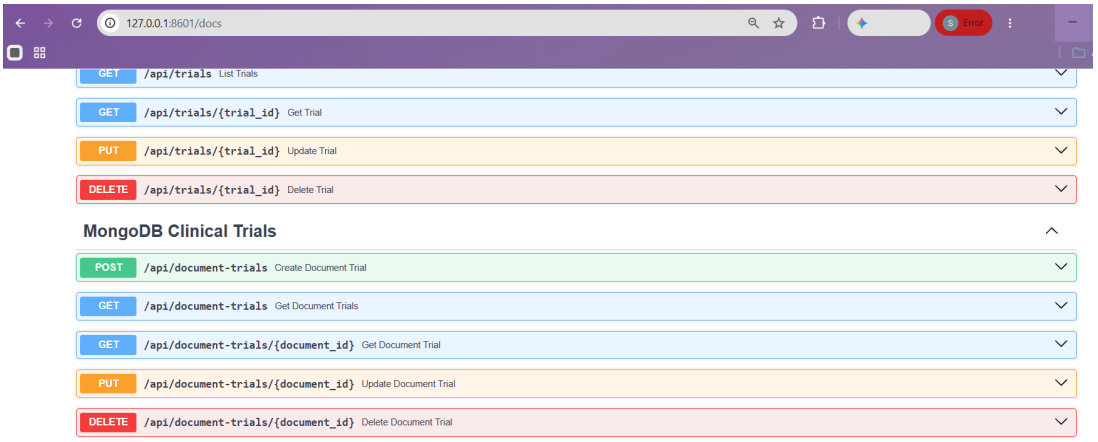


Figure 1. MongoDB clinical-trial CRUD operations and schema-validation evidence.

The application successfully completed create, read, update, and delete operations. MongoDB validation rejected invalid clinical-trial documents. The complete raw evidence is stored in reports/hw06/raw/part1_crud_results.json.

Part 2: Conversational Memory

The conversational-memory implementation uses three MongoDB collections:

Collection	Purpose
messages	Stores user and assistant conversation turns
summaries	Stores session and lifetime summaries
episodes	Stores durable facts and their embedding vectors

Short-term memory retrieves recent conversation turns. Session summaries preserve older information from one session, while lifetime summaries combine information across multiple sessions. Episodic memory uses sentence-transformer embeddings and cosine similarity to retrieve durable facts relevant to the current request.

The FastAPI application exposes:

- POST /api/chat
- GET /api/memory/{user_id}
- GET /api/aggregate/{user_id}

Ten-Message Demonstration

The required demonstration submitted ten user messages. The system stored each user message and corresponding assistant response.

Metric	Result
User messages submitted	10
Total stored messages	20
Recent messages returned	16
Session summary created	Yes
Lifetime summary created	Yes
Episodic memories created	10
Daily aggregate messages	20

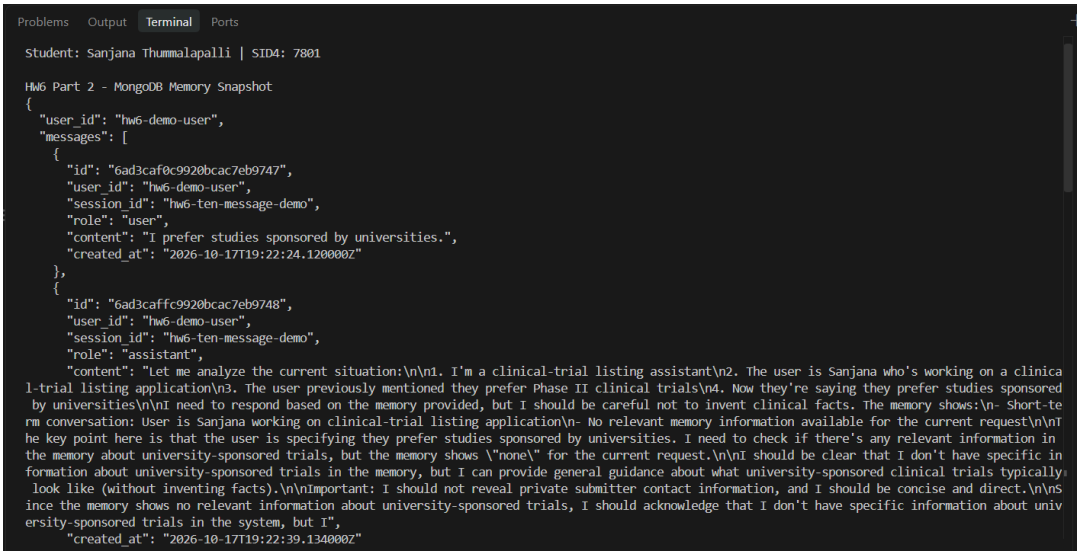


Figure 2. Persisted messages, summaries, and episodic memories from the ten-message demonstration.

Part 3: Memory Quality and Cost Experiment

A fixed conversation containing 30 messages was evaluated using five memory probes. The same conversation, probes, local model, and scoring rules were used for every configuration.
The evaluated configurations were:

- SHORT_TERM_N=4
- SHORT_TERM_N=12
- SHORT_TERM_N=30
- Compression policy

The experiment measured mean prompt tokens, mean model latency, estimated cost per 10,000 conversations, and mean probe quality from 1 to 5.
Results

Configuration	**Mean Prompt Tokens**	**Mean Latency (ms)**	**Probe Quality**	**Cost per 10,000**
SHORT_TERM_N=4	869.433	12,144.905	3.8	$500.83
SHORT_TERM_N=12	1,479.400	17,831.934	4.2	$683.82
SHORT_TERM_N=30	1,059.467	12,478.687	1.4	$557.84
Compression policy	934.167	12,065.704	3.8	$520.25

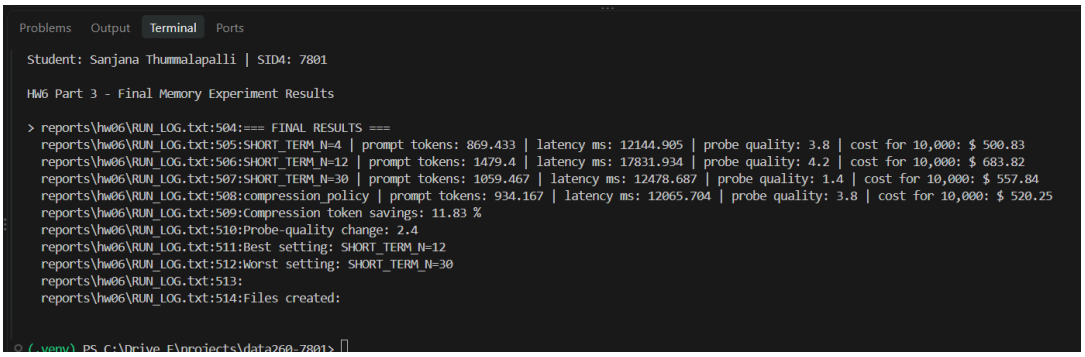


Figure 3. Prompt-token, latency, quality, compression, and estimated-cost results for the four memory configurations.

Analysis
SHORT_TERM_N=12 produced the highest mean probe-quality score of 4.2. It retained sufficient recent context to answer most memory probes correctly. However, this configuration also produced the largest prompt, highest latency, and highest estimated cost.

SHORT_TERM_N=4 reduced mean prompt size and latency while maintaining a quality score of 3.8. This demonstrates that summaries and episodic retrieval can compensate for a smaller recent-message window.
SHORT_TERM_N=30 produced the weakest quality score of 1.4. Increasing the nominal short-term history did not automatically improve memory. Competing information and the model context limit reduced the usefulness of the additional history.

The compression policy achieved 11.83% prompt-token savings while retaining a quality score of 3.8. It also had the lowest mean latency of 12,065.704 milliseconds.
For maximum memory quality, SHORT_TERM_N=12 is recommended. For a production system balancing quality, latency, and cost, the compression policy is recommended.

Part 4: File-Based Memory
The existing AGENT.md was expanded with durable instructions covering five categories:

- Project description
- Technology stack
- Coding conventions
- Testing approach
- Prohibited actions

The filter_trials_by_phase function was implemented and tested against these instructions. The function uses type hints, descriptive snake_case names, a public-function docstring, input validation, and a predictable response structure. It does not expose credentials or access the local model directly.

MEMORY.md Commands
The file-based memory implementation supports:

Command	**Purpose**
/memory	Display current memory and consolidation status
/compact	Consolidate memory and report before/after counts
/dream	Extract durable facts from recent conversation
/help	Explain the available commands

Consolidation Test
The original MEMORY.md contained 55 fact lines. It intentionally included six exact duplicates and three contradictory fact pairs.

The contradictions concerned:

1. Phase II versus Phase III preference
2. San Jose versus Santa Clara location preference
3. Five-message versus ten-message summary interval

The /compact command merged duplicate facts and explicitly recorded how every contradiction was resolved. The consolidation reduced the file from 55 lines to 35 lines. The /dream command subsequently added one durable fact, leaving a final total of 36 lines.

Both versions were preserved:

- reports/hw06/raw/MEMORY_original.md
- reports/hw06/raw/MEMORY_consolidated.md

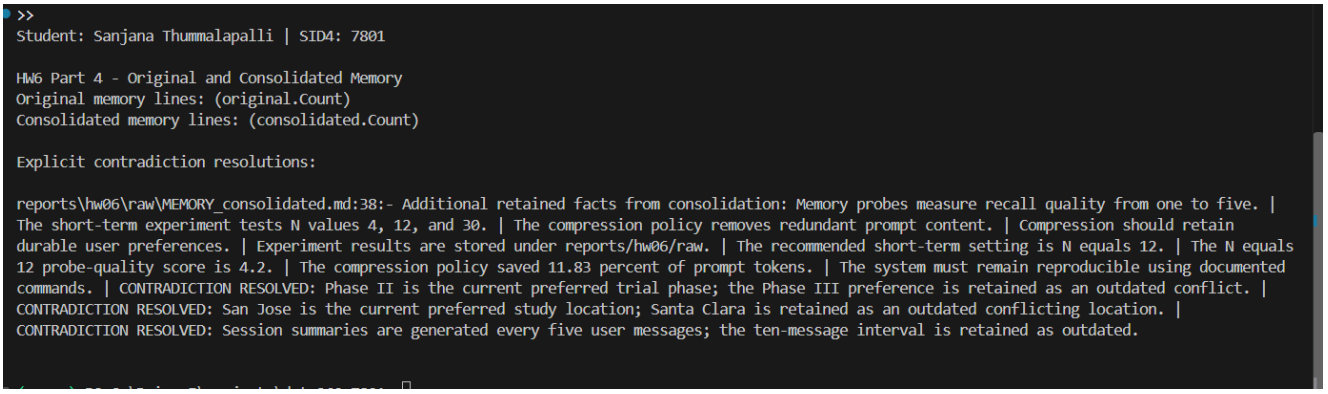


Figure 4. File-memory consolidation reduced 55 durable facts below 40 lines and explicitly recorded three contradiction resolutions.

Comparison of Memory Systems

Criterion	**MongoDB Memory**	**File-Based Memory**
Inspectability	Requires MongoDB queries or API access	Directly readable in a text editor
Restart persistence	Persists through the Docker volume	Persists directly in repository files
Multi-session scalability	Strong	Limited
Multi-user support	Strong through indexed identifiers	Requires manual separation
Structured retrieval	Supports filtering and aggregation	Limited
Critical instructions	May depend on retrieval	Always placed in the prompt
Version history	Requires separate audit mechanisms	Naturally supported by Git

MongoDB is better for scalable conversational history, multi-user separation, aggregation, and episodic retrieval. File-based memory is better for essential instructions, safety constraints, and facts that must consistently remain visible.

The best solution is a hybrid system. AGENT.md stores stable behavior and project rules, MEMORY.md stores a compact set of critical durable facts, and MongoDB stores scalable conversation history.

Final Verification

The final verification checked:

- Required files
- Supported Python version
- AGENT.md instructions
- Original and consolidated memory
- File-memory commands
- AGENT.md-compliant function
- FastAPI routes
- MongoDB collections
- Experiment case counts
- Raw JSON files
- Experiment configurations
- Reproducibility log

All automated checks passed.

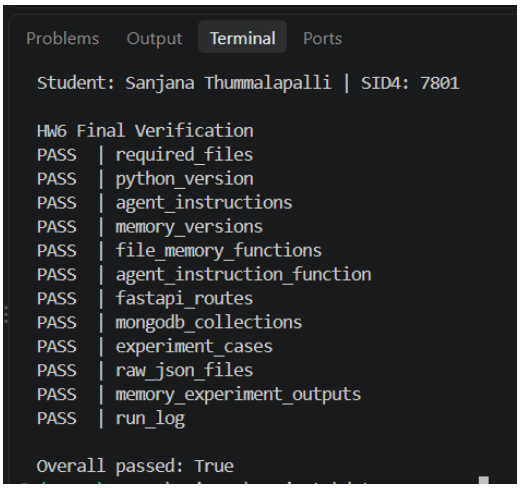


Figure 5. Final Homework 6 verification showing all required checks passing.

AI Use Disclosure

- ChatGPT/Codex was used to interpret the assignment, draft implementation components, debug runtime problems, organize experiments, and prepare documentation.
- The local qwen3:4b Ollama model was used for conversational responses, summaries, memory probes, and memory-policy experiments.
- All AI-assisted code was reviewed and executed. I tested MongoDB, Docker, FastAPI endpoints, memory persistence, experiment output, consolidation behavior, and final verification. The complete disclosure is available in reports/hw06/AI_USE.md.

Conclusion

Homework 6 successfully added MongoDB document storage, persistent conversational memory, summarization, episodic retrieval, memory-cost experimentation, and file-based durable memory to the Clinical Trial Listing application.

The experiment demonstrated that a larger context window does not automatically produce better memory quality. SHORT_TERM_N=12 produced the best probe quality, while the compression policy offered the strongest balance of memory quality, latency, and prompt cost.

The final hybrid design combines MongoDB scalability with the readability, version history, and durability of Markdown-based memory.